

PHBSpline: Technical Implementation Specification

Dynamic planar Pythagorean-hodograph B-splines with verified distance-domain access

Prepared from the cubic-ph-spline 1.1.0 reference implementation

2026-08-05

Contents

Document status	4
Executive specification	4
1. Scope	5
1.1 In scope	5
1.2 Explicitly out of scope for version 1	5
2. Baseline compatibility and intentional differences	6
2.1 API compatibility objective	6
2.2 Baseline design principles that SHALL be retained	6
2.3 Intentional differences	6
2.4 Repository source audit and reuse plan	7
3. Mathematical model	8
3.1 Complex planar representation	8
3.2 Logical B-spline preimage	8
3.3 Local span coordinate	9
3.4 Bernstein product formula	9
3.5 Bernstein antiderivative	9
4. Continuity contract and degree selection	10
4.1 Constructor continuity parameters	10
4.2 Degree rule	10
4.3 Geometric continuity verification	11
4.4 Parametric continuity verification	11
4.5 Curvature derivatives	11
5. Input domain and geometry policy	11
5.1 Accepted point configurations	11
5.2 Invalid point data	12
5.3 Nonconsecutive duplicates	12
5.4 Shape policy	12
6. Public API	12
6.1 Constructor	12
6.2 Required properties	13
6.3 Scalar geometry methods	13
6.4 Distance methods	15
6.5 Batch methods	15
6.6 Editing methods	16
6.7 Snapshots	17

6.8 Stable handles and locations	17
7. Policy dataclasses	17
7.1 ConstructionPolicy	17
7.2 EditingPolicy	17
7.3 InversePolicy	18
7.4 NumericalPolicy	18
8. Internal architecture	18
8.1 Layer separation	18
8.2 Recommended module layout	19
8.3 Dynamic tree	19
8.4 Compiled span kernel	20
9. Coordinate normalization and scaled representation	20
9.1 No mutable global spatial normalization	20
9.2 Patch frame	20
9.3 Span frame	20
9.4 Endpoint anchoring	21
10. Initial construction	21
10.1 Construction pipeline	21
10.2 Parameter weights	21
10.3 Guide construction	21
10.4 Square-root branch initializer	22
10.5 Exact interpolation constraints	22
10.6 Analytic Jacobian	22
10.7 Shape objective	23
10.8 Nonlinear method	23
10.9 Minimum-degree hidden-knot allocation	23
11. Local editing	23
11.1 Transaction model	23
11.2 Locality invariant	24
11.3 Patch size	24
11.4 Move operation	24
11.5 Insert operation	24
11.6 Delete operation	25
11.7 Edit complexity contract	25
12. Regularity certification	25
12.1 Required property	25
12.2 Bernstein convex-hull certificate	25
12.3 Relative regularity margin	26
12.4 Speed evaluation	26
13. Exact forward arc-length compilation	26
13.1 Authoritative metric	26
13.2 Forward and reverse arc polynomials	26
13.3 Polynomial evaluation strategy	27
13.4 Length aggregation	27
14. Random distance access	27
14.1 Global algorithm	27
14.2 Per-span inverse LUT	27
14.3 Monotone cubic seed	28
14.4 Safeguarded correction	28
14.5 Endpoint reversal	29

14.6 Residual tolerance	29
14.7 Global length resolution	29
15. Geometry evaluation	29
15.1 Position	29
15.2 Arbitrary-order derivative vectors	29
15.3 Tangent	30
15.4 Curvature and curvature-vector derivatives	30
15.5 Normals	31
15.6 Offsets	31
16. Numerical safety requirements	31
16.1 General prohibitions	31
16.2 Safe norms and differences	32
16.3 FMA and compensated arithmetic	32
16.4 Underflow and overflow	32
16.5 Small-angle formulas	32
16.6 Determinism	32
16.7 High-order geometry evaluation	32
17. Verification pipeline	33
17.1 Independent verification principle	33
17.2 Required constructor checks	33
17.3 Position tolerance	33
17.4 Continuity tolerance	33
17.5 Verification report	34
18. Exceptions	34
18.1 Hierarchy	34
18.2 Structured fields	35
19. Complexity requirements	35
20. Serialization	35
20.1 Authoritative state	35
20.2 Loading	36
21. Threading and concurrency	36
22. Testing specification	36
22.1 Baseline test migration	36
22.2 Input geometry matrix	36
22.3 Continuity matrix	37
22.4 Dynamic-edit property tests	37
22.5 Distance inversion tests	37
22.6 Arbitrary-order geometry queries	37
22.7 High-precision oracle tests	38
22.8 Scale and transformation invariance	38
22.9 Fuzzing	38
22.10 Performance regression tests	38
23. Acceptance criteria	39
24. Implementation sequence	39
Phase 1 - immutable mathematical core	39
Phase 2 - distance kernel	39
Phase 3 - static PH B-spline constructor	39
Phase 4 - dynamic tree and local edits	40

Phase 5 - enterprise hardening	40
25. Example usage	40
26. Required pseudocode	41
26.1 Scalar distance query	41
26.2 Strict-local move	42
27. Design rationale and nonclaims	42
27.1 Why quintic is the default	42
27.2 Why the inverse is numerical despite polynomial arc length	42
27.3 Why local support needs patch closure	42
27.4 Why strict locality can fail	43
28. References	43
Appendix A. Suggested diagnostic dataclasses	43
Appendix B. Double-double primitives	44
Appendix C. Formal derivative and curvature-vector jet recurrence	44
Appendix D. Benchmark reporting template	45

Document status

Status: implementation specification, revision 0.3.

Target: a new `PHBSpline` class and supporting package architecture derived from the numerical and API principles of [ahrvoje/cubic-ph-spline](#), but generalized from immutable cubic PH segments to mutable, variable-order planar PH B-splines.

Reviewed baseline: repository `main` as retrieved on 2026-08-05; package metadata reports version 1.1.0. The retrieved test suite completed with **491 passed** under the review environment. The baseline package documents normalized construction, independently verified nonlinear solves, exact polynomial arc length, cancellation-resistant cubic inversion, and approximately 5-6 microseconds per scalar random `point_at_length` query for 100 through 10,000 segments on its reported benchmark platform.

This document is normative. It is intended to be handed directly to an implementation team.

The terms **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY**, and **OPTIONAL** are to be interpreted as requirement levels.

Executive specification

`PHBSpline` SHALL be a mutable planar point-interpolating spline with the following principal properties:

1. It accepts an ordered sequence of finite planar points in arbitrary geometric configuration: convex, nonconvex, inflectional, self-intersecting, looping, backtracking, or containing nonconsecutive repeated points. Consecutive coincident points are rejected by default because they do not define a nonzero interpolation span.
2. It represents a piecewise polynomial Pythagorean-hodograph curve using a complex polynomial B-spline preimage. Its speed and cumulative arc length are piecewise polynomials constructed analytically, without numerical quadrature.
3. It supports any requested finite continuity orders `g_order`, `c_order`, and `curvature_order`, subject to configured degree and resource limits. If none are supplied, the default guarantee is G2.
4. It supports exact interpolation-point move, insertion, and deletion as atomic local patch operations. In strict-local mode, the work is bounded independently of total point count and the operation either commits a verified patch or raises and rolls back. In expanding mode, the patch may grow and worst-case work may become linear in the number of points.

5. It provides an API adapted from CubicPHSpline: `point`, `tangent`, `normal`, `principal_normal`, `signed_curvature`, `curvature_vector`, `arc_length`, `parameter_at_length`, and `point_at_length`, plus arbitrary-order derivative-vector and curvature-vector queries, batch queries, and dynamic-edit APIs.
6. Random distance access uses an augmented length tree, a per-span inverse lookup kernel, and safeguarded Halley/Newton/ITP correction. The result is accepted only after a forward arc-length residual test near binary64 precision.
7. Every constructor and edit operation is transactional. Solver success flags are never sufficient. Interpolation, continuity, PH reconstruction, regularity, arc-length monotonicity, and inverse-kernel postconditions are independently verified before commit.
8. The internal curve, hodograph, speed, arc-length polynomial, tangent, normal, curvature, and PH offset data are constructively derived from the preimage coefficients. The interpolation coefficients themselves are generally obtained by a deterministic nonlinear solve, not by a universal symbolic formula.

The implementation SHALL NOT claim unconditional existence of a regular fixed-degree PH interpolant for every possible input and every requested order. Its contract is:

For every valid input, construction either returns a spline satisfying all declared guarantees and verification bounds, or raises a typed exception containing structured diagnostic information. It never returns an unverified or silently downgraded curve.

1. Scope

1.1 In scope

The first production version SHALL implement:

- open and closed planar splines;
- ordered point interpolation;
- variable finite continuity requirements;
- a polynomial PH representation;
- dynamic point move, insertion, deletion, append, and prepend;
- local transactional reconstruction;
- exact analytic forward arc length;
- efficient scalar and vectorized distance-domain evaluation;
- arbitrary-order parameter and arc-length derivative vectors and curvature-vector derivatives;
- deterministic binary64 numerics with explicit failure modes;
- immutable query snapshots;
- typed exceptions and diagnostics;
- persistence of the authoritative interpolation data and verified compiled representation;
- test and benchmark infrastructure capable of detecting numerical and asymptotic regressions.

1.2 Explicitly out of scope for version 1

The following are not required for the first implementation:

- spatial or quaternion PH splines;
- rational PH input curves;
- PH surfaces;
- global minimum-bending-energy guarantees;
- universal loop-free or self-intersection-free interpolation;
- automatic obstacle avoidance;
- exact symbolic inverse of a generic high-degree arc polynomial;
- a proof that every arbitrary point sequence admits a regular solution under a fixed hidden-span cap;
- GPU kernels;
- exact arithmetic construction;
- a public position-space control polygon with the same semantics as a NURBS control polygon.

These omissions MUST be documented in the public package.

2. Baseline compatibility and intentional differences

2.1 API compatibility objective

The following scalar methods SHALL retain the baseline naming and result conventions:

```
point(u) -> np.ndarray           # shape (2,), dtype float64
tangent(u) -> np.ndarray          # unit vector
normal(u, side="left") -> np.ndarray
principal_normal(u) -> np.ndarray
signed_curvature(u) -> float
curvature_vector(u) -> np.ndarray
arc_length(u) -> float
parameter_at_length(s) -> float
point_at_length(s) -> np.ndarray
```

The baseline `curvature_vector(u)` call SHALL remain source-compatible and mean order zero. PHBSpline SHALL additionally expose named arbitrary-order vector queries:

```
derivative(u, order=1, *, wrt="parameter", side="auto") -> np.ndarray
curvature_vector(u, order=0, *, wrt="arc_length", side="auto") -> np.ndarray
```

These methods are not numerical-differentiation conveniences. They are required geometry kernels with elementary coefficient formulas, scale-aware error control, and the same strict failure behavior as distance and curvature evaluation.

Scalar arguments SHALL accept Python and NumPy real scalars, reject booleans, arrays, sequences, NaN, and infinities, and apply only a documented few-ULP endpoint clamp. Array input belongs to explicit batch methods.

2.2 Baseline design principles that SHALL be retained

The new package SHALL retain these principles from `cubic-ph-spline`:

- normalized construction rather than raw-coordinate nonlinear solving;
- deterministic initializers and bounded iteration counts;
- independent acceptance checks after every nonlinear solve;
- no generic numerical quadrature for arc length;
- no generic geometric search for `point_at_length`;
- cancellation-resistant endpoint handling;
- explicit regularity margins;
- structured exceptions carrying index, quantity, measured value, and required bound;
- warnings treated as test failures;
- extensive adversarial scale, curvature, chord-ratio, and round-trip tests;
- immutable compiled segment kernels even when the owning spline is mutable.

2.3 Intentional differences

PHBSpline differs from CubicPHSpline in the following mandatory ways:

Concern	CubicPHSpline baseline	PHBSpline requirement
Primitive degree	Cubic PH segments	Degree deduced from continuity request; default quintic PH
Arc inverse	Elementary monotone cubic inverse	Cached monotone inverse plus safeguarded polynomial correction
Object mutability	Immutable	Mutable, transactional, with immutable snapshots
Metric index	Flat compensated prefix array	Dynamic augmented tree; optional compact snapshot array
Nonconvex handling	Specialized cubic preprocessing	No convexity admissibility restriction; generic regular PH solve

Concern	CubicPHBSpline baseline	PHBSpline requirement
Continuity	G2 on convex runs, G1 at selected transitions	User-selected finite G, C, and curvature continuity, verified at every join
Geometry queries	Tangent and order-zero curvature vector	Arbitrary-order parameter/arc-length derivative vectors and curvature-vector derivatives
Local editing	Not supported	Move/insert/delete with bounded local patch mode
Representation	Cubic Bezier plus linear preimage	Variable-order B-spline preimage plus per-span Bernstein kernels

2.4 Repository source audit and reuse plan

The implementation plan is based on a direct review of the repository source and tests, not only its README. The following mapping SHALL guide reuse and replacement:

Baseline file	Observed responsibility	Requirement for PHBSpline
<code>ph_spline/cubic.py</code>	Immutable public API, scalar validation, global parameter dispatch, prefix-length lookup, construction post-verification	Retain scalar semantics and verification style; split mutable ownership from immutable snapshot querying; replace flat prefixes with a dynamic tree
<code>segment.py</code>	Dual cubic Bezier and linear complex-preimage storage; point, tangent, speed, curvature, and local inverse methods	Generalize to variable-degree immutable <code>SpanKernel</code> with Bernstein preimage, exact speed/arc controls, and compiled numerical inverse
<code>arclength.py</code>	FMA Horner forms, cancellation-aware cubic arc evaluation, scaled hyperbolic/Cardano estimate, endpoint-reversed inversion, bounded safeguarded Newton, explicit residual gate	Retain forward/reverse evaluation, fixed iteration bounds, residual verification, and endpoint exactness; replace the cubic estimator by the LUT seed and generic monotone polynomial correction
<code>construction.py</code>	Input validation, normalization, geometric planning, segment assembly	Retain validation and normalized local solving; replace convex-run/cubic-specific planning with generic preimage B-spline constraints and hidden-knot refinement
<code>nonlinear.py</code>	Deterministic bounded nonlinear solve with structured fallback and independent acceptance	Retain determinism, analytic/complex-step verification, damping, and hard bounds; generalize to banded equality-constrained PH displacement solves
<code>exceptions.py</code>	Value/runtime dual inheritance and structured diagnostics	Preserve the pattern, expand fields for point IDs, span IDs, edit operation, and patch
<code>tests/</code>	Arc inversion, extreme scale, frames, G2, nonconvex data, invariants, straight cases, and validation	Port all behavioral intents and add variable-order, dynamic-edit, tree, and 100,000-point tests

The existing `CubicPHSpline` class SHALL remain importable. The package top-level `ph_spline.__init__` SHALL export `PHSpline`, `CubicPHSpline`, and `PHBSpline`; adding the new class MUST NOT make either concrete implementation inherit from the other.

3. Mathematical model

3.1 Complex planar representation

Represent a planar point by

$$z(t) = x(t) + iy(t).$$

A planar polynomial PH curve is generated by a complex polynomial or spline preimage $w(t)$:

$$z'(t) = w(t)^2.$$

Its speed is

$$\sigma(t) = |z'(t)| = |w(t)|^2 = w(t)\overline{w(t)},$$

and its arc length is

$$S(t) = \int \sigma(t) dt.$$

If w is a degree- m polynomial on a knot span, then:

$$\deg z = 2m + 1, \quad \deg \sigma = 2m, \quad \deg S = 2m + 1.$$

3.2 Logical B-spline preimage

The logical preimage is

$$w(t) = \sum_j c_j N_{j,m}(t), \quad c_j \in \mathbb{C},$$

on a nondecreasing knot sequence. The implementation MAY store the curve in segmented Bezier-extracted form, but the authoritative pieces MUST be compatible with a B-spline preimage and MUST pass left/right preimage-jet verification at every knot.

With degree m and interior knot multiplicity ν , generically:

$$w \in C^{m-\nu}, \quad z \in C^{m-\nu+1}, \quad \kappa \in C^{m-\nu-1}.$$

The default construction SHALL use simple interior knots, $\nu = 1$, unless a specific lower continuity is explicitly requested as part of a future extension. Thus:

$$w \in C^{m-1}, \quad z \in C^m, \quad \kappa \in C^{m-2}.$$

3.3 Local span coordinate

For a knot span $[\tau_i, \tau_{i+1}]$, define

$$h_i = \tau_{i+1} - \tau_i > 0, \quad u = \frac{t - \tau_i}{h_i} \in [0, 1].$$

Let the Bezier-extracted preimage be

$$w_i(\nu) = \sum_{a=0}^m b_{i,a} B_a^m(\nu).$$

Then

$$\frac{dz}{d\nu} = h_i w_i(\nu)^2, \quad \frac{ds}{d\nu} = h_i |w_i(\nu)|^2.$$

All runtime span kernels SHALL use $\nu \in [0, 1]$. No evaluator SHALL form a high-degree global power polynomial in an unscaled global parameter.

3.4 Bernstein product formula

For degree- m Bernstein coefficients b_a , the degree- $2m$ hodograph coefficients are

$$q_k = h_i \sum_{a+b=k} \frac{\binom{m}{a} \binom{m}{b}}{\binom{2m}{k}} b_a b_b, \quad k = 0, \dots, 2m.$$

The speed coefficients are

$$r_k = h_i \sum_{a+b=k} \frac{\binom{m}{a} \binom{m}{b}}{\binom{2m}{k}} b_a \bar{b}_b.$$

The imaginary part of each computed r_k SHALL be checked against a rounding bound and discarded only after that check. Silent conversion of a materially complex speed coefficient to real is forbidden.

3.5 Bernstein antiderivative

For a degree- d Bernstein polynomial

$$f(\nu) = \sum_{k=0}^d a_k B_k^d(\nu),$$

an antiderivative with $F(0) = 0$ has coefficients

$$A_0 = 0, \quad A_{k+1} = A_k + \frac{a_k}{d+1}, \quad k = 0, \dots, d.$$

This formula SHALL generate both the position and arc-length coefficients. Numerical quadrature SHALL NOT be used to produce authoritative span lengths.

4. Continuity contract and degree selection

4.1 Constructor continuity parameters

The constructor SHALL expose:

```
g_order: int | None = None
c_order: int | None = None
curvature_order: int | None = None
```

Semantics:

- `g_order=r` requests oriented geometric continuity G^r .
- `c_order=r` requests parametric continuity C^r in the package global parameter.
- `curvature_order=k` requests signed curvature continuity $\kappa \in C^k$ with derivatives taken with respect to arc length.
- `None` means that continuity type is not independently requested.
- If all three are `None`, the constructor SHALL set `g_order=2`.
- Boolean values are invalid despite `bool` being an `int` subclass.
- Valid orders are nonnegative integers subject to the configured resource limit.

4.2 Degree rule

Define

$$r_* = \max(2, g_{\text{req}}, c_{\text{req}}, k_{\text{req}} + 2),$$

where a missing request contributes zero. The base preimage degree SHALL be

$$m = r_*,$$

and the curve degree SHALL be

$$p = 2m + 1.$$

The minimum $m = 2$ is intentional. It gives quintic PH spans and avoids the fixed-curvature-sign restriction of regular planar PH cubics.

The implementation SHALL use exactly $m = r_*$. Shape freedom and numerical regularity SHALL be obtained by inserting simple knots, not by increasing the polynomial degree beyond the continuity requirement. The object MUST report both the selected preimage degree and curve degree.

The reference construction inserts one simple knot at the parameter midpoint of every user interval. Consequently, each user interval contains two compiled polynomial spans. A simple knot preserves generic C^{m-1} preimage continuity while supplying one additional complex control degree of freedom per interval; exact PH displacement constraints retain interpolation at the user knots.

Examples:

Requested guarantee	Preimage degree m	Curve degree $2m + 1$	Generic simple-knot result
default G2	2	5	at least C2, curvature C0
C3	3	7	C3, curvature C1
G4	4	9	at least C4, curvature C2
curvature C2	4	9	C4, curvature C2
C5, curvature C3	5	11	C5, curvature C3

4.3 Geometric continuity verification

For a regular oriented planar curve parameterized by arc length s :

$$T_s = i\kappa T.$$

The verifier SHALL treat G^r as matching:

$$T_- = T_+,$$

and, for $r \geq 2$,

$$\frac{d^j \kappa_-}{ds^j} = \frac{d^j \kappa_+}{ds^j}, \quad j = 0, \dots, r-2.$$

If stronger parametric C^r continuity is produced by the B-spline construction, that is acceptable and SHALL be reported as an actual guarantee.

4.4 Parametric continuity verification

For `c_order=r`, the implementation SHALL verify physical left/right derivatives with respect to the global parameter through order r :

$$\frac{d^j z_-}{dt^j} = \frac{d^j z_+}{dt^j}, \quad j = 0, \dots, r.$$

The verifier SHALL account for span widths h_i ; comparing unscaled local derivatives is incorrect.

4.5 Curvature derivatives

Curvature derivatives SHALL be defined with respect to arc length unless explicitly requested otherwise:

$$D_s = \frac{1}{\sigma(t)} D_t.$$

The implementation SHALL compute curvature jets using formal truncated power-series arithmetic or an algebraically equivalent recurrence. Finite differences are forbidden for continuity acceptance.

5. Input domain and geometry policy

5.1 Accepted point configurations

The input is an **ordered** point sequence, not an unordered point cloud. Subject to finite representability and nonzero consecutive chords, the implementation SHALL accept attempts to construct splines through:

- convex and nonconvex polylines;
- any number of inflections;
- self-intersecting polylines;
- curves that must loop to satisfy continuity;
- exact or near direction reversals;
- alternating very short and very long chords;
- nonconsecutive repeated points;
- open near-closed paths;
- closed paths.

No convexity, simplicity, orientation, monotone-turn, or star-shaped predicate may be a constructor precondition.

5.2 Invalid point data

The constructor SHALL reject:

- fewer than two points for an open curve;
- fewer than three distinct cyclic anchors for a closed curve;
- malformed arrays or elements;
- booleans as coordinates;
- NaN or infinite coordinates;
- consecutive points that compare equal after conversion to binary64;
- a chord whose physical displacement or length cannot be represented reliably under the numerical policy;
- an input whose requested continuity order exceeds the configured resource limit.

For a closed curve, an exactly duplicated final point equal to the first MAY be accepted and canonicalized by removing the duplicate. Tolerance-based point merging is forbidden.

5.3 Nonconsecutive duplicates

Nonconsecutive duplicate points SHALL be retained as distinct interpolation constraints with distinct point handles. They may imply a loop. They MUST NOT be silently deduplicated.

5.4 Shape policy

The default policy SHALL prioritize:

1. exact interpolation;
2. requested continuity;
3. regularity;
4. deterministic construction;
5. low preimage strain and curvature variation;
6. short and visually local deviations from the input polyline.

It SHALL NOT promise a simple curve. Optional policies MAY request `avoid_self_intersection`, `prefer_convex`, or `limit_turning`, but these are extra constraints that can cause a typed construction failure.

6. Public API

6.1 Constructor

The required signature is:

```
class PHBSpline:
    def __init__(
        self,
        points: ArrayLike,
        *,
        closed: bool = False,
        g_order: int | None = None,
        c_order: int | None = None,
        curvature_order: int | None = None,
        construction: ConstructionPolicy | None = None,
        editing: EditingPolicy | None = None,
        inverse: InversePolicy | None = None,
        numerics: NumericalPolicy | None = None,
    ) -> None: ...
```

Policies SHALL be immutable dataclasses. `None` selects documented defaults.

6.2 Required properties

```
@property
def points(self) -> NDArray[np.float64]: ...           # read-only snapshot copy

@property
def point_handles(self) -> tuple[PointHandle, ...]: ...

@property
def num_points(self) -> int: ...

@property
def num_spans(self) -> int: ...                       # includes hidden spans

@property
def preimage_degree(self) -> int: ...

@property
def degree(self) -> int: ...                          #  $2*m + 1$ 

@property
def requested_continuity(self) -> ContinuitySpec: ...

@property
def verified_continuity(self) -> ContinuitySpec: ...

@property
def closed(self) -> bool: ...

@property
def length(self) -> float: ...

@property
def length_coordinate(self) -> LengthCoordinate: ...

@property
def version(self) -> int: ...

@property
def diagnostics(self) -> BuildDiagnostics: ...

@property
def last_edit_report(self) -> EditReport | None: ...
```

points MUST return a read-only snapshot copy and MUST NOT expose mutable storage. Its cost is $O(N)$; high-frequency consumers SHALL retain a snapshot or use point handles rather than repeatedly requesting the full array.

6.3 Scalar geometry methods

```
def point(self, u: Real) -> NDArray[np.float64]: ...
def derivative(
    self,
    u: Real,
    order: int = 1,
    *,
    wrt: Literal["parameter", "arc_length"] = "parameter",
    side: Literal["auto", "left", "right"] = "auto",
) -> NDArray[np.float64]: ...
```

```

def tangent(self, u: Real) -> NDArray[np.float64]: ...
def normal(self, u: Real, side: Literal["left", "right"] = "left") -> NDArray[np.float64]: ...
def principal_normal(self, u: Real) -> NDArray[np.float64]: ...
def signed_curvature(self, u: Real) -> float: ...
def curvature_vector(
    self,
    u: Real,
    order: int = 0,
    *,
    wrt: Literal["arc_length", "parameter"] = "arc_length",
    side: Literal["auto", "left", "right"] = "auto",
) -> NDArray[np.float64]: ...
def curvature_derivative(
    self,
    u: Real,
    order: int = 1,
    *,
    wrt: Literal["arc_length", "parameter"] = "arc_length",
    side: Literal["auto", "left", "right"] = "auto",
) -> float: ...
def jet(
    self,
    u: Real,
    order: int,
    *,
    wrt: Literal["parameter", "arc_length"] = "parameter",
    side: Literal["auto", "left", "right"] = "auto",
) -> tuple[NDArray[np.float64], ...]: ...
def curvature_vector_jet(
    self,
    u: Real,
    order: int,
    *,
    wrt: Literal["arc_length", "parameter"] = "arc_length",
    side: Literal["auto", "left", "right"] = "auto",
) -> tuple[NDArray[np.float64], ...]: ...

```

`derivative(u, r, wrt=q)` returns the vector $D_q^r z$ for any nonnegative integer r permitted by the configured evaluation-order resource limit. That limit is `NumericalPolicy.max_evaluation_order`. Order zero is exactly `point(u)`. The parameter derivative is with respect to the package global compatibility parameter, not the local span coordinate. The arc-length derivative satisfies `derivative(u, 1, wrt="arc_length") == tangent(u)` within its certified rounding bound. The tangent remains a unit-vector convenience and MUST NOT be used as a substitute for the generally nonunit first parameter derivative.

Let $C_0 = \kappa N_L$ be the ordinary curvature vector. `curvature_vector(u, j, wrt=q)` returns $D_q^j C_0$, not merely $(D_q^j \kappa) N_L$. Consequently,

$$C_j^{(s)} = D_s^{j+2} z.$$

The no-keyword call `curvature_vector(u)` retains the `CubicPHSpline` order-zero result. `jet(..., order=r)` returns $(z, D_q z, \dots, D_q^r z)$, and `curvature_vector_jet(..., order=j)` returns $(C_0, D_q C_0, \dots, D_q^j C_0)$. A full jet SHALL share one recurrence and workspace; it MUST NOT invoke the single-order method repeatedly.

`order` SHALL accept Python and NumPy integer scalars, reject booleans and nonintegral values, and be nonnegative. An order above the configured resource limit raises `ResourceLimitError`. A parameter derivative whose order exceeds the local curve degree is the exact zero vector. Arc-length derivatives and curvature-vector derivatives do not generally terminate with polynomial degree.

At a join, `side="left"` or `side="right"` requests that one-sided value. With `side="auto"`, the evaluator SHALL

return a common value only when the relevant left/right jets agree within their independently computed error bounds; otherwise it raises `DiscontinuousDerivativeError`. At an open endpoint, `auto` selects the interior side and an explicitly unavailable side is invalid. Away from a join, `side` does not change the result.

The global compatibility parameter `u` is in `[0, 1]`. It is derived from the dynamic parameter-weight tree. A local geometry edit can therefore change the normalized `u` assigned to downstream geometry even when that geometry is spatially unchanged. Applications requiring edit-stable references SHALL use `CurveLocation` or `PointHandle`.

For an open spline, `u=0` and `u=1` are the two endpoints. For a closed spline, `u=0` and `u=1` denote the same seam point and frame; `arc_length(0)=0` and `arc_length(1)=length`. Exact seam queries SHALL not choose inconsistent left/right branches.

6.4 Distance methods

```
def arc_length(self, u: Real, *, extended: bool = False) -> float | LengthCoordinate: ...
def parameter_at_length(self, s: Real | LengthCoordinate) -> float: ...
def location_at_length(self, s: Real | LengthCoordinate) -> CurveLocation: ...
def point_at_length(self, s: Real | LengthCoordinate) -> NDArray[np.float64]: ...
def frame_at_length(self, s: Real | LengthCoordinate) -> Frame2D: ...
def distance_between(
    self,
    u0: Real,
    u1: Real,
    *,
    mode: Literal["absolute", "signed", "forward"] = "absolute",
) -> float: ...
def advance_by_length(self, location: CurveLocation, ds: Real) -> CurveLocation: ...
def point_after_length(self, location: CurveLocation, ds: Real) -> NDArray[np.float64]: ...
```

`advance_by_length` is REQUIRED because a global binary64 distance cannot resolve every small local increment when total length is extremely large. It SHALL perform relative tree traversal without first forming `global_s + ds` in one float. `point_after_length(location, ds)` SHALL be equivalent to `point(advance_by_length(location, ds))`; the name deliberately avoids confusion with a geometric normal offset curve.

`distance_between` semantics are exact and SHALL be:

- `mode="signed"`: `arc_length(u1) - arc_length(u0)` on the chosen `[0, 1]` cut;
- `mode="absolute"`: the absolute value of the signed result;
- `mode="forward"`: traversal-direction distance from `u0` to `u1`, adding total length on a closed curve when the target precedes the source, and rejecting backward travel on an open curve.

6.5 Batch methods

```
def points_at(self, u: ArrayLike, *, out: NDArray[np.float64] | None = None) ->
    NDArray[np.float64]: ...
def tangents_at(self, u: ArrayLike, *, out: NDArray[np.float64] | None = None) ->
    NDArray[np.float64]: ...
def derivatives_at(
    self,
    u: ArrayLike,
    order: int = 1,
    *,
    wrt: Literal["parameter", "arc_length"] = "parameter",
    side: Literal["auto", "left", "right"] = "auto",
    out: NDArray[np.float64] | None = None,
) -> NDArray[np.float64]: ...
def curvature_vectors_at(
    self,
    u: ArrayLike,
```

```

    order: int = 0,
    *,
    wrt: Literal["arc_length", "parameter"] = "arc_length",
    side: Literal["auto", "left", "right"] = "auto",
    out: NDArray[np.float64] | None = None,
) -> NDArray[np.float64]: ...
def points_at_length(self, s: ArrayLike, *, out: NDArray[np.float64] | None = None,
    ↪ assume_sorted: bool = False) -> NDArray[np.float64]: ...
def parameters_at_length(self, s: ArrayLike, *, out: NDArray[np.float64] | None = None,
    ↪ assume_sorted: bool = False) -> NDArray[np.float64]: ...

```

For sorted distances, the implementation SHALL use a linear span walk rather than one tree search per query.

The derivative batch methods accept one common order and return `u.shape + (2,)`. They SHALL group queries by span, reuse compiled derivative ladders, and avoid scalar Python dispatch per element. Their scalar validation, join-side semantics, and numerical acceptance rules are identical to the corresponding scalar methods.

6.6 Editing methods

```

def point_handle(self, index: int) -> PointHandle: ...
def index_of(self, handle: PointHandle) -> int: ...

def move_point(
    self,
    point: int | PointHandle,
    value: ArrayLike,
    *,
    repair: EditRepair | None = None,
) -> EditReport: ...

def insert_point(
    self,
    index: int,
    value: ArrayLike,
    *,
    repair: EditRepair | None = None,
) -> InsertResult: ...

def delete_point(
    self,
    point: int | PointHandle,
    *,
    repair: EditRepair | None = None,
) -> EditReport: ...

def append_point(self, value: ArrayLike, *, repair: EditRepair | None = None) -> InsertResult:
    ↪ ...
def prepend_point(self, value: ArrayLike, *, repair: EditRepair | None = None) -> InsertResult:
    ↪ ...

def edit(self, *, repair: EditRepair | None = None) -> EditTransaction: ...

```

Insertion index semantics SHALL match `list.insert`: insert before the current item at `index`; `index == num_points` appends.

Every mutation SHALL be atomic. On any exception, input points, point handles, geometry, metric tree, version, and caches SHALL remain exactly as before the operation.

6.7 Snapshots

```
def snapshot(self, *, compact: bool = False) -> PHBSplineSnapshot: ...
```

A snapshot SHALL be immutable and thread-safe for concurrent reads. It SHALL expose the scalar and batch query API but no mutation API.

- `compact=False` MAY use persistent shared tree nodes and shall be at most $O(\log N)$ after a local edit if persistent nodes are used.
- `compact=True` SHALL build contiguous span arrays and a flat compensated prefix array in $O(N)$, optimized for repeated static queries.

6.8 Stable handles and locations

```
@dataclass(frozen=True, slots=True)
```

```
class PointHandle:
```

```
    id: int
```

```
@dataclass(frozen=True, slots=True)
```

```
class CurveLocation:
```

```
    span_id: int
```

```
    local_u: float
```

```
    version: int
```

```
@dataclass(frozen=True, slots=True)
```

```
class LengthCoordinate:
```

```
    hi: float
```

```
    lo: float
```

A deleted `PointHandle` SHALL raise `StaleHandleError`. A `CurveLocation` from an older version SHALL raise `StaleLocationError` unless the implementation can prove that its span identity and local geometry were preserved.

7. Policy dataclasses

The package SHALL expose policies at public or advanced-public scope. Defaults below are normative starting values and MAY be tuned only with benchmark and test evidence.

7.1 ConstructionPolicy

```
@dataclass(frozen=True, slots=True)
```

```
class ConstructionPolicy:
```

```
    parameterization: Literal["centripetal", "chord", "uniform"] = "centripetal"
```

```
    shape_objective: Literal["preimage_strain", "guide_fairness"] = "preimage_strain"
```

```
    max_iterations: int = 48
```

```
    max_line_search_steps: int = 16
```

```
    max_hidden_spans_per_input_span: int = 8
```

```
    max_refinement_rounds: int = 6
```

```
    initial_trust_radius: float = 0.25
```

```
    interpolation_weight: float = 1.0
```

```
    guide_weight: float = 1.0
```

```
    strain_weight: float = 1.0e-3
```

```
    speed_variation_weight: float = 1.0e-4
```

```
    deterministic: bool = True
```

Random initialization is forbidden in deterministic mode.

7.2 EditingPolicy

```
@dataclass(frozen=True, slots=True)
```

```
class EditingPolicy:
```

```

default_repair: Literal["strict_local", "expand", "global"] = "strict_local"
initial_patch_spans: int | None = None          # None -> 2*(m + 3)
max_patch_spans: int = 64
expansion_factor: float = 2.0
leaf_capacity: int = 128
preserve_outside_bitwise: bool = True

```

`strict_local` SHALL never silently perform a global rebuild. `expand` may grow the patch up to `max_patch_spans`; after that it fails. `global` may rebuild the entire spline.

7.3 InversePolicy

```

@dataclass(frozen=True, slots=True)
class InversePolicy:
    lut_nodes_min: int = 8
    lut_nodes_max: int = 128
    lut_power_of_two: bool = True
    seed_kind: Literal["monotone_cubic", "linear"] = "monotone_cubic"
    fast_iterations: int = 2
    max_iterations: int = 67
    use_halley: bool = True
    fallback: Literal["itp", "bisection"] = "itp"
    endpoint_reverse_threshold: float = 0.5

```

7.4 NumericalPolicy

```

@dataclass(frozen=True, slots=True)
class NumericalPolicy:
    dtype: Literal["float64"] = "float64"
    regularity_ratio_min: float = 1.0e-12
    max_preimage_degree: int = 16
    max_evaluation_order: int = 64
    max_regularization_subdivision_depth: int = 24
    parameter_ulp_slack: int = 4
    position_eps_factor: float = 256.0
    tangent_abs_tol: float = 1.0e-12
    curvature_rel_tol: float = 1.0e-10
    continuity_eps_factor: float = 1024.0
    inverse_eps_factor: float = 64.0
    use_longdouble_verification: Literal["auto", "never", "always"] = "auto"
    reject_unresolved_global_lengths: bool = True

```

Requests requiring a preimage degree above `max_preimage_degree` SHALL raise `ResourceLimitError` before construction begins. Users MAY explicitly provide a higher limit.

`max_evaluation_order` is a resource guard, not a mathematical restriction on the API. Users MAY explicitly increase it. Implementations SHALL reject an order before allocating order-dependent work when it exceeds the configured limit.

8. Internal architecture

8.1 Layer separation

The package SHALL separate four layers:

1. **Interpolation model:** user points, point handles, continuity request, policies, and optional Hermite constraints.
2. **Authoritative PH spline:** knot topology and complex preimage coefficients or equivalent compatible span data.

3. **Compiled span kernels:** position, preimage, speed, arc length, inverse LUT, regularity bounds, and bounding boxes.
4. **Dynamic indexes:** order-statistic point tree, parameter tree, and arc-length tree.

A query SHALL never invoke the nonlinear construction solver.

8.2 Recommended module layout

```
ph_spline/
  __init__.py
  api.py           # PHBSpline and PHBSplineSnapshot
  policies.py      # immutable policy dataclasses
  types.py         # handles, locations, frames, diagnostics
  exceptions.py
  validation.py
  knots.py         # knot generation, insertion, removal, extraction
  bernstein.py     # product, integration, subdivision, bounds
  preimage.py      # complex B-spline operations
  construction.py  # initial build orchestration
  guide.py         # local guide and square-root initializer
  solver.py        # deterministic sparse/banded SQP
  continuity.py    # jet generation and verification
  regularity.py    # nonvanishing certificates
  span.py          # immutable compiled span kernel
  inverse.py       # inverse LUT and safeguarded solve
  length.py        # double-double arithmetic and tree aggregates
  tree.py          # persistent B+ tree / rope
  editing.py       # local transactions
  snapshot.py
  serialization.py
  _constants.py
  py.typed
```

Circular imports SHALL be avoided. `span.py`, `inverse.py`, and `length.py` SHALL not import the mutable API layer.

8.3 Dynamic tree

The mutable object SHALL use a balanced B+ tree, rope, or equivalent order-statistic tree. Each internal node SHALL aggregate:

- number of user points;
- number of PH spans;
- parameter-weight sum as double-double;
- arc-length sum as double-double;
- subtree bounding box;
- subtree version/hash information sufficient for persistent snapshots.

Each leaf SHALL contain a bounded block, default 128 spans or an implementation-equivalent block size.

Stable point IDs are not ordered by spline position. The implementation SHALL therefore maintain a point-ID locator table mapping each live `PointHandle.id` to its current leaf record. Hash lookup may be $O(1)$ average; deriving the current sequence index from that record and parent subtree counts is $O(\log N)$. Leaf split, merge, and compaction operations SHALL update all affected locator entries before commit.

Required complexities for fixed continuity order and bounded patch size:

- locate point handle or index: $O(\log N)$;
- locate parameter span: $O(\log N)$;
- locate distance span: $O(\log N)$;
- update aggregates after a local edit: $O(\log N)$;

- split or merge a leaf: $O(\log N)$ amortized.

A linked list of spans plus a flat prefix array is insufficient for the mutable class because every local length change would require updating all downstream prefixes.

8.4 Compiled span kernel

Each immutable `SpanKernel` SHALL contain at least:

```
span_id
left_point_id / right_point_id or hidden-span metadata
parameter_width h
local affine frame (origin, power-of-two scale, optional unit rotation)
preimage degree m
preimage Bernstein coefficients b[0:m+1]
position Bernstein coefficients p[0:2m+2]
speed Bernstein coefficients r[0:2m+1]
arc Bernstein coefficients a[0:2m+2]
forward power coefficients for arc residual evaluation
reverse power coefficients for endpoint-reversed evaluation
span length as double-double and rounded float64
regularity lower and upper bounds
derivative and curvature-vector jet cache data
inverse LUT
axis-aligned bounding box
verification digest / version
```

The arrays SHALL be read-only after construction.

9. Coordinate normalization and scaled representation

9.1 No mutable global spatial normalization

A single global origin and longest-chord scale, while effective for an immutable object, is unsuitable as the only dynamic representation: moving one point far outside the original scale can force an $O(N)$ rebase.

The mutable implementation SHALL therefore solve and compile bounded patches in local affine frames.

9.2 Patch frame

For each solve patch, choose:

- origin equal to a fixed patch boundary point or a safe midpoint;
- spatial scale $H = 2^e$, selected with `frexp/ldexp` so normalized chords are near unity;
- an optional unit complex rotation that maps a representative chord toward the positive real axis.

Power-of-two scaling is REQUIRED because it introduces no additional rounding in binary64 exponent adjustment.

The normalized solve SHALL aim to keep all active point coordinates, preimage controls, and derived polynomial coefficients within a moderate exponent range. If that cannot be achieved, raise `NumericalPrecisionError` rather than allow overflow or underflow.

9.3 Span frame

A span MAY retain its patch frame or be recompiled into its own local frame. Physical evaluation SHALL use

$$z(\nu) = O_i + H_i R_i \hat{z}_i(\nu),$$

where O_i is the local origin, H_i a power-of-two scale, and $|R_i| = 1$.

Physical speed and curvature scale as

$$\sigma = H_i \hat{\sigma}, \quad \kappa = \hat{\kappa}/H_i.$$

All cross-frame join comparisons SHALL be transformed to physical scale before verification.

9.4 Endpoint anchoring

Every span SHALL store exact references to its structural endpoints. `point(0)`, `point(1)`, and exact interpolation-knot queries SHALL return the stored input point values.

This endpoint return is permitted only after the independently evaluated polynomial displacement residual satisfies the interpolation tolerance. Endpoint snapping MUST NOT conceal a failed solve.

10. Initial construction

10.1 Construction pipeline

The constructor SHALL execute the following stages:

1. validate and canonicalize input;
2. determine continuity request and degree;
3. assign stable point handles;
4. generate initial knot/parameter weights;
5. construct a local ordinary guide curve;
6. derive a coherent complex square-root preimage initializer;
7. solve the exact PH displacement constraints with a deterministic constrained nonlinear method;
8. insert one simple midpoint knot in every user interval;
9. compile all span kernels;
10. independently verify all postconditions;
11. build the dynamic point, parameter, and distance trees;
12. publish the object only after all stages succeed.

No partially initialized public object may escape.

10.2 Parameter weights

For chord lengths $d_i > 0$, initial span weights SHALL be:

- uniform: $h_i = 1$;
- chord: $h_i \propto d_i$;
- centripetal: $h_i \propto \sqrt{d_i}$.

Weights SHALL be computed with exponent-scaled norms and normalized using compensated summation. No positive weight may round to zero. If the dynamic range prevents a strictly increasing binary64 compatibility parameter, the tree SHALL retain extended sums and `CurveLocation` remains authoritative.

10.3 Guide construction

The guide exists only to choose a good PH branch and fairness target. It is not authoritative geometry.

For each interpolation point, the implementation SHALL use a bounded local stencil and one of:

- scaled local polynomial least squares solved by QR with column pivoting;
- centripetal Catmull-Rom derivatives with explicit reversal fallback;
- another deterministic local method with equivalent tested robustness.

The guide SHALL provide tangent direction and speed samples at preimage Greville abscissae. Near a zero guide derivative, the implementation SHALL choose a deterministic fallback direction from adjacent nonzero chords. At an exact reversal, it SHALL choose a side using local signed area, the previous coherent branch, or a fixed documented tie-breaker. It SHALL NOT divide by an unguarded tangent norm.

10.4 Square-root branch initializer

For a guide derivative $g'(t) = v(t)e^{i\theta(t)}$, initialize

$$w(t) \approx \sqrt{v(t)}e^{i\theta(t)/2}.$$

Angles SHALL be unwrapped before halving. The sign of each square root SHALL be selected to minimize the distance to the previous preimage sample, producing a coherent branch. Closed curves require a cyclic sign-consistency check; if the branch closes with opposite sign, the initializer SHALL insert or choose a branch transition compatible with the curve hodograph, not silently introduce a discontinuity.

The samples SHALL be fitted to B-spline controls using a scaled banded least-squares solve.

10.5 Exact interpolation constraints

For each input interval with displacement

$$D_i = P_{i+1} - P_i,$$

the constraint is

$$F_i(c) = \int_{\tau_i}^{\tau_{i+1}} w(t)^2 dt - D_i = 0.$$

In local span coordinates:

$$F_i(c) = \sum_{j \in \mathcal{S}_i} \int_0^1 h_j w_j(\nu)^2 d\nu - D_i,$$

where $\mathcal{S}_i$ contains the visible and hidden spans between the two user anchors.

These are two real equations per input interval. Their support is local in the preimage control ordering.

10.6 Analytic Jacobian

For a local Bezier preimage coefficient $b_k = x_k + iy_k$:

$$\frac{\partial F}{\partial x_k} = 2h \int_0^1 w(\nu) B_k^m(\nu) d\nu,$$

$$\frac{\partial F}{\partial y_k} = 2ih \int_0^1 w(\nu) B_k^m(\nu) d\nu.$$

For global B-spline controls, multiply by the Bezier extraction matrix. All integrals SHALL be evaluated exactly through Bernstein products and coefficient sums.

Finite-difference Jacobians are forbidden in the production solver. Complex-step differentiation MAY be retained as an independent debug/test oracle, as in the baseline package.

10.7 Shape objective

The constrained solver SHALL minimize a deterministic local objective while satisfying the displacement constraints. The default objective SHALL include:

$$J(c) = \lambda_g \|W_g(c - c_0)\|_2^2 + \lambda_s \int |D_t^2 w(t)|^2 dt + \lambda_v J_{\text{speed}}(c).$$

The first term retains the guide branch. The second is an exact quadratic preimage-strain energy. `J_speed` MAY use fixed-order Gauss-Legendre samples because it is a shape objective, not authoritative arc-length computation.

The objective MUST NOT override interpolation, continuity, or regularity constraints.

10.8 Nonlinear method

The recommended method is deterministic sparse sequential quadratic programming or equality-constrained trust-region Gauss-Newton. At each iteration solve a scaled KKT system of the form

$$\begin{bmatrix} H + \mu I & J^T \\ J & 0 \end{bmatrix} \begin{bmatrix} \delta c \\ \delta \lambda \end{bmatrix} = - \begin{bmatrix} \nabla_c \mathcal{L} \\ F \end{bmatrix}.$$

Requirements:

- variables SHALL be ordered by knot support so the matrix has fixed bandwidth for fixed degree;
- the factorization SHALL exploit banded or sparse structure;
- row and column scaling SHALL be applied;
- the trust-region or damping parameter SHALL be bounded and deterministic;
- every step SHALL be checked for finite values;
- line search SHALL use a fixed maximum number of steps;
- no solver flag is sufficient for acceptance;
- the iteration count SHALL be hard bounded;
- the final candidate SHALL pass the independent verifier.

For fixed degree, bounded refinement, and bounded iterations, initial construction is $\mathcal{O}(N)$ in point count.

10.9 Minimum-degree hidden-knot allocation

The base topology SHALL contain one simple midpoint knot in every input interval. This fixed allocation supplies the degrees of freedom needed for exact displacement interpolation while retaining the minimum preimage degree $m = r_*$. All extraction SHALL be performed directly from endpoint basis jets; an ill-conditioned Bernstein collocation inverse is not acceptable for high-order requests.

If this topology cannot satisfy the interpolation residual or regularity certificate, construction SHALL raise the corresponding typed error. It MUST NOT increase m , reduce continuity, or accept a near-cusp. A future adaptive simple-knot refinement policy MAY add further local knots without changing m , provided the actual hidden-span count is reported.

11. Local editing

11.1 Transaction model

Every edit SHALL operate on a private candidate tree and candidate span kernels. Commit consists of one atomic root/version replacement. On failure, the candidate is discarded.

A transaction may batch multiple point changes:

```
with curve.edit(repair="strict_local") as tx:
    tx.move_point(handle_a, [x1, y1])
    tx.insert_point(index_b, [x2, y2])
    tx.delete_point(handle_c)
```

The union of affected neighborhoods SHALL be solved once when practical.

11.2 Locality invariant

Let a patch cover the parameter interval $[a, b]$. A committed local edit SHALL satisfy:

- all preimage coefficients and span kernels outside the patch are unchanged;
- required preimage or curve jets at the patch boundaries match the frozen exterior data;
- the patch boundary positions are fixed unless an endpoint of the whole open spline is intentionally moved;
- the total patch displacement equals the new boundary displacement.

For a purely interior point move, exterior boundary points are unchanged, so

$$\int_a^b (w_{\text{new}}(t)^2 - w_{\text{old}}(t)^2) dt = 0.$$

This closure condition prevents a local hodograph change from translating the entire downstream curve.

If `preserve_outside_bitwise=True`, unchanged span objects SHALL be structurally shared and therefore bitwise identical.

11.3 Patch size

The initial patch SHALL include:

- every input interval whose displacement constraint changed;
- every span whose basis support intersects a variable preimage control;
- any hidden spans belonging to those input intervals.

With one midpoint knot per input interval, an interior patch with both exterior jets fixed needs at least m logical intervals for constraint closure. The reference default uses $m + 3$ logical intervals, hence $2(m + 3)$ compiled spans. It therefore rebuilds 10, 14 and 22 spans for G2, G4 and G8 respectively. The count depends on requested order; a fixed count independent of m is not a valid general rule.

11.4 Move operation

`move_point` SHALL:

1. resolve the stable handle or index;
2. validate the new point against adjacent points;
3. update only the two adjacent displacement constraints for an interior open point, one for an open endpoint, or two cyclic constraints for a closed curve;
4. construct a candidate local guide and preimage seed;
5. freeze exterior controls/jets;
6. solve and verify the patch;
7. recompile affected spans and inverse kernels;
8. update tree aggregates on the path to the root;
9. commit and increment `version`.

11.5 Insert operation

`insert_point(index, value)` SHALL:

1. identify the old input interval being split;
2. assign a new stable point ID;
3. choose an initial parameter split from adjacent chord lengths with scaled arithmetic;
4. insert the corresponding preimage knot exactly, preserving the old curve as a seed;
5. replace one displacement constraint by two;
6. solve and verify a bounded patch;
7. split tree leaves if necessary;

8. commit atomically.

Insertion MUST NOT renumber existing point handles.

11.6 Delete operation

`delete_point` SHALL:

1. reject deletion if it would leave too few points;
2. merge the two adjacent displacement constraints into one;
3. remove or mark the interpolation knot and generate a seed using stable knot removal or local least-squares projection;
4. solve and verify the bounded patch;
5. merge underfull tree leaves when appropriate;
6. invalidate the deleted handle only after commit.

11.7 Edit complexity contract

For fixed continuity order, fixed degree, fixed maximum hidden spans, and `strict_local` repair:

$$T_{\text{edit}} = O(\log N) + O(Km^3I),$$

where K is bounded by `max_patch_spans` and I by the iteration cap. Therefore the edit is $O(\log N)$ with respect to total point count. `C_eval(m)` denotes the span-evaluation cost: conservatively $O(m^2)$ for de Casteljau and $O(m)$ for an accepted stable Horner/Bernstein evaluator. All advertised scaling in N assumes the requested continuity order, and hence m , is fixed.

This bound is obtained by allowing failure. If the strict patch has no verified solution, `LocalEditFailure` SHALL be raised and the object SHALL be unchanged.

For `expand`, the patch grows geometrically. Typical work remains local, but worst-case complexity is $O(N)$. For `global`, $O(N)$ is explicit.

No documentation may claim unconditional $O(1)$ or $O(\log N)$ successful editing for arbitrary point displacement without stating this policy distinction.

12. Regularity certification

12.1 Required property

Every committed span SHALL satisfy

$$|w(t)| > 0$$

throughout the closed span, with a quantitative margin. Sampling alone is insufficient.

12.2 Bernstein convex-hull certificate

A Bezier polynomial lies in the convex hull of its control points. For each preimage span:

1. compute the convex hull of the complex Bernstein controls in $\mathbb{R}^2$;
2. compute the distance d from the origin to that hull;
3. if $d > 0$ by a certified rounding margin, then $|w(\nu)| \geq d$;
4. otherwise subdivide the preimage by de Casteljau at $\nu = 1/2$ and repeat;
5. stop when all subspans are certified or the subdivision-depth limit is reached.

If the origin remains inside or numerically indistinguishable from the hull at the depth limit, construction SHALL fail. The exception is `NonRegularSplineError`.

12.3 Relative regularity margin

Let $d_{\min}$ be the certified lower bound and $d_{\max}$ an upper bound from the maximum control radius after subdivision. Define

$$\rho = \frac{d_{\min}^2}{d_{\max}^2}.$$

The span SHALL satisfy

$$\rho > \rho_{\min},$$

with default $\rho_{\min} = 10^{-12}$. This protects distance inversion and curvature evaluation from near-cuspidal conditioning.

Users MAY request a stricter margin. Weakening the default requires an explicit advanced policy and SHALL be reflected in diagnostics.

12.4 Speed evaluation

Runtime speed SHALL be evaluated from the preimage,

$$\sigma = h|w|^2,$$

not solely from a potentially cancellation-prone expanded speed polynomial. Use scaled complex evaluation and hypot-style normalization before squaring.

13. Exact forward arc-length compilation

13.1 Authoritative metric

For every span, the speed and arc length SHALL be constructed by exact polynomial coefficient operations from the preimage. No adaptive quadrature, tessellation, or chord-length approximation may contribute to the authoritative length.

13.2 Forward and reverse arc polynomials

Each span SHALL compile:

$$S_f(\nu) = \int_0^\nu h|w(v)|^2 dv,$$

and

$$S_r(v) = \int_0^v h|w(1-q)|^2 dq = L - S_f(1-v).$$

S_r SHALL be compiled directly from reversed coefficients. Near the right endpoint, inversion and residual checks SHALL use S_r and target $L - s$, never a subtraction of two nearly equal forward lengths.

13.3 Polynomial evaluation strategy

Each span SHALL store Bernstein arc coefficients and MAY store power coefficients for speed. Evaluation SHALL use this hierarchy:

1. FMA Horner on forward or reverse power coefficients when the compiled condition estimate is acceptable;
2. compensated Horner if FMA is unavailable;
3. de Casteljau on Bernstein coefficients when cancellation or overflow risk exceeds policy;
4. scaled exponent arithmetic if a physical scale is extreme.

The implementation SHALL precompute a conservative absolute evaluation-error bound.

13.4 Length aggregation

Span lengths SHALL be aggregated in double-double form using error-free transformations such as `TwoSum` and `FastTwoSum`. Every tree aggregate stores `(hi, lo)` with

$$\text{value} = hi + lo.$$

The compatibility `length` property returns the correctly rounded or best available binary64 `hi + lo`. `length_coordinate` returns both parts.

14. Random distance access

14.1 Global algorithm

`location_at_length(s)` SHALL perform:

1. validate and canonicalize `s` as float or `LengthCoordinate`;
2. search the augmented length tree for the containing span in $O(\log N)$;
3. compute the local target without catastrophic cancellation using double-double subtraction;
4. normalize to $\xi = s_{\text{local}}/L \in [0, 1]$;
5. select a per-span LUT cell in constant time;
6. evaluate a monotone inverse seed;
7. apply safeguarded Halley or Newton correction;
8. verify the forward or reverse arc residual;
9. return `CurveLocation(span_id, local_u, version)`.

`point_at_length` then evaluates one span kernel.

14.2 Per-span inverse LUT

For normalized arc map

$$F(\nu) = \frac{S_f(\nu)}{L}, \quad F : [0, 1] \rightarrow [0, 1],$$

build a table at construction time with uniformly spaced distance nodes

$$\xi_j = \frac{j}{M}, \quad j = 0, \dots, M,$$

where M is a power of two between policy limits. For each node store:

- ξ_j ;
- the verified inverse ν_j ;
- inverse slope

$$\frac{d\nu}{d\xi} = \frac{L}{h|w(\nu_j)|^2};$$

- optional second-derivative data;
- the exact bracket $[\nu_j, \nu_{j+1}]$.

Uniform distance nodes permit cell selection by a multiply and integer clamp. Node inverses SHALL be generated sequentially with a bracketed ITP or bisection/Newton solve, reusing the previous node as the next lower bracket; every node is accepted by the same forward/reverse residual rule used at runtime.

The table size SHALL be increased until the seed-quality criterion is met or the maximum is reached. For each cell, compile certified Bernstein bounds

$$m_1 \leq F'(u), \quad |F''(u)| \leq M_2.$$

A recommended fast-path criterion is

$$\frac{M_2 \Delta u}{m_1} \leq 0.25,$$

where Δu is the cell bracket width, together with successful one-correction tests at the cell midpoint and two interior Chebyshev points. The inequality is a conditioning/performance criterion, not a correctness requirement; failure at `lut_nodes_max` SHALL mark the span as `slow_inverse` and retain the fully safeguarded fallback.

14.3 Monotone cubic seed

The default seed SHALL be a monotone cubic Hermite interpolation of $\nu(\xi)$. Endpoint slopes SHALL be limited by a Fritsch-Carlson or equivalent monotonicity rule. The seed MUST lie within the table bracket; otherwise it is clamped to the bracket midpoint and a diagnostic counter is incremented.

The LUT is an accelerator, not an authority. An inaccurate seed cannot compromise correctness because the subsequent solver remains bracketed.

14.4 Safeguarded correction

Let

$$f(\nu) = S(\nu) - s_{\text{local}}, \quad f'(\nu) = h|w(\nu)|^2,$$

and compute f'' analytically from the preimage. The Halley proposal is

$$\nu_H = \nu - \frac{2ff'}{2(f')^2 - ff''}.$$

The proposal SHALL be rejected in favor of Newton if:

- any operand is nonfinite;
- the denominator is nonpositive or too small by a scaled bound;
- the step leaves the current bracket;
- the predicted step is not a descent step.

The Newton proposal is

$$\nu_N = \nu - \frac{f}{f'}.$$

It SHALL be rejected in favor of ITP or bisection if nonfinite, outside the bracket, or if the evaluated residual fails to decrease.

The bracket SHALL be updated after every evaluation. Iteration SHALL have a hard maximum. Failure to satisfy the residual bound raises `ArcLengthInversionError`; an unverified parameter is never returned.

14.5 Endpoint reversal

If $s_{\text{local}} > L/2$, use

$$v = 1 - \nu, \quad S_r(v) = L - s_{\text{local}}.$$

Recover $\nu = 1 - v$ only after the reversed solve. Exact endpoints SHALL return 0.0 or 1.0 without iteration.

14.6 Residual tolerance

Let ϵ be binary64 machine epsilon and E_S the conservative arc-evaluation error bound. Accept only if

$$|S(\nu) - s_{\text{local}}| \leq 64\epsilon L + 4 \text{ulp}(s_{\text{local}}) + 2E_S.$$

For reversed evaluation, use the corresponding reversed target and error bound.

The parameter error estimate SHALL be reported or internally bounded by

$$|\delta\nu| \lesssim \frac{\text{arc residual}}{\min f'}.$$

14.7 Global length resolution

A binary64 scalar cannot distinguish two global distances separated by less than one ULP of the accumulated prefix. The implementation SHALL detect this condition.

When policy rejects unresolved global lengths, an ambiguous scalar query SHALL raise `LengthResolutionError` and recommend:

- `LengthCoordinate` input;
- `advance_by_length` from a local location;
- a compact local subcurve.

It SHALL NOT silently select an arbitrary neighboring span.

15. Geometry evaluation

15.1 Position

Position SHALL be evaluated with de Casteljau or a numerically equivalent stable Bezier method in the local span frame. Exact structural endpoints return stored anchors after verification.

15.2 Arbitrary-order derivative vectors

Let a compiled span have curve degree $n = 2m + 1$ and local position Bernstein coefficients $p_0, \dots, p_n$. For $0 \leq r \leq n$, the elementary Bezier derivative formula is

$$D_\nu^r z(\nu) = \frac{n!}{(n-r)!} \sum_{a=0}^{n-r} \Delta^r p_a B_a^{n-r}(\nu),$$

and $D_\nu^r z = 0$ for $r > n$. For $r \geq 1$, the PH representation gives the algebraically equivalent identity

$$D_\nu^r z(\nu) = h \sum_{j=0}^{r-1} \binom{r-1}{j} D_\nu^j w(\nu) D_\nu^{r-1-j} w(\nu).$$

The span compiler SHALL construct derivative Bernstein ladders in its normalized local frame. Runtime evaluation SHALL use scaled de Casteljau, or a demonstrably equivalent compensated Bernstein evaluator, on those ladders.

The first derivative SHALL use the direct hw^2 expression. For higher orders, the compiler SHALL retain the direct curve-control ladder and MAY retain the preimage Leibniz ladder; the runtime kernel SHALL use the route with the tighter precomputed forward-error bound. Finite differences, generic polynomial differentiation at query time, and conversion to a high-degree global power basis are forbidden.

For the global B-spline parameter t , the local affine map gives

$$D_t^r z = h^{-r} D_\nu^r z.$$

If the compatibility parameter is normalized by the parameter-weight total, its additional affine factor SHALL be applied by the dispatcher. Products such as h^{-r} and falling factorials MUST NOT be formed naively. Accumulate scale factors as mantissa plus binary exponent and restore them with checked `ldexp`; use ratio recurrences so an overflowing intermediate cannot corrupt a representable final result.

For intrinsic derivatives, let

$$\beta(\nu) = \frac{ds}{d\nu} = h|w(\nu)|^2, \quad D_s = \beta^{-1} D_\nu.$$

The evaluator SHALL form normalized Taylor coefficients and apply the recurrence

$$Z_0 = z, \quad Z_{r+1} = \beta^{-1} D_\nu Z_r,$$

as truncated series through the requested order. Series multiplication and reciprocal are finite elementary coefficient operations. Normalized coefficients (derivative divided by factorial) SHALL be used internally to avoid factorial and binomial overflow. One request for an order- r jet SHALL cost bounded local work, normally $O(mr + r^2)$ arithmetic and $O(r)$ workspace, independent of total spline size.

15.3 Tangent

Evaluate the preimage $w = a + ib$, scale it by its norm, and square the normalized complex number:

$$T = \left(\frac{w}{|w|} \right)^2.$$

To reduce cancellation, compute

$$T_x = (r - s)(r + s), \quad T_y = 2rs,$$

where $r = a/|w|$, $s = b/|w|$. Renormalize only when the squared norm differs from one by more than a documented rounding threshold. This optimized kernel SHALL agree with `derivative(u, 1, wrt="arc_length")` within the combined error bounds.

15.4 Curvature and curvature-vector derivatives

With derivatives taken with respect to the global B-spline parameter t , define

$$A(t) = \text{Im}(\overline{w(t)} w'(t)), \quad B(t) = |w(t)|^2.$$

The signed curvature is the elementary rational expression

$$\kappa(t) = 2A(t)B(t)^{-2}.$$

In complex-vector form, multiplication by i rotates left by 90 degrees, so the order-zero curvature vector has the direct expression

$$C_0(t) = \kappa N_L = 2A(t) i w(t)^2 B(t)^{-3} = D_s^2 z(t).$$

For $q \in \{t, s\}$, the requested higher-order curvature vector is

$$C_j^{(q)} = D_q^j C_0.$$

For **wrt="parameter"**, construct truncated series for A , B , w^2 , and B^{-1} , then obtain the requested vector by finite series multiplication. For **wrt="arc_length"**, apply $D_s = B^{-1} D_t$, with the required local/global scale conversion, or reuse the curve arc-length jet through order $j+2$. The latter path SHALL satisfy $C_j^{(s)} = D_s^{j+2} z$ by construction and is preferred when it has the tighter error bound. **curvature_derivative** uses the same $2AB^{-2}$ series but returns the scalar derivative; a curvature-vector derivative is not formed by multiplying that scalar by a fixed normal.

Preimage derivatives SHALL be evaluated from Bernstein forward differences. Taylor reciprocal SHALL begin only after scaled evaluation has certified $B(0) > 0$ consistently with the span regularity bound. Coefficient convolutions SHALL use FMA accumulation when available and compensated pairwise accumulation otherwise. All powers of the preimage scale, span width, and physical frame scale SHALL be tracked by exponent arithmetic and restored only once. A certified straight span returns an exact zero curvature vector for order zero and all orders; a nonstructural near-zero result retains its sign and direction and MUST NOT be snapped to zero.

The scalar curvature and order-zero vector kernels MAY use shorter specialized formulas, but they SHALL share coefficient conventions and error bounds with the arbitrary-order kernel. Curvature at an inflection is exactly or numerically zero; **principal_normal** is undefined when curvature is zero within the curvature-zero bound and SHALL raise **UndefinedPrincipalNormalError**. The curvature vector itself remains defined and equals zero at an exact inflection.

15.5 Normals

The left normal is

$$N_L = (-T_y, T_x),$$

and the right normal is $-N_L$. The principal normal is

$$N_P = \text{sign}(\kappa) N_L.$$

15.6 Offsets

Exact rational PH offsets are a valuable CAD consequence but are OPTIONAL for version 1. If implemented, an offset SHALL be generated from

$$z_d(t) = z(t) + d N(t)$$

using the rational unit normal from the PH representation. It MUST preserve the same knot topology and SHALL document offset singularities where $1 - d\kappa = 0$.

16. Numerical safety requirements

16.1 General prohibitions

Production code MUST NOT:

- trust a nonlinear optimizer's success flag without independent checks;
- use unbounded loops;
- use random starts by default;
- use finite differences for accepted continuity jets;

- use finite differences for any public derivative, curvature derivative, or curvature-vector query;
- use generic polynomial root solvers in the distance hot path;
- use numerical quadrature for authoritative arc length;
- subtract nearly equal forward cumulative lengths near a span endpoint;
- square unscaled values that may overflow;
- accept NaN or infinity through a clamp;
- silently reduce continuity, degree, or regularity requirements;
- silently rebuild globally when strict-local editing was requested;
- compare geometry with one absolute tolerance independent of scale and conditioning.
- form high-order factorials, binomial coefficients, reciprocal span-width powers, or preimage powers without checked scaling.

16.2 Safe norms and differences

Use scaled `hypot` algorithms for vector norms. Coordinate differences SHALL be computed after safe exponent scaling when direct subtraction risks overflow. If the actual displacement is not representable in binary64 user coordinates, raise `NumericalPrecisionError`.

16.3 FMA and compensated arithmetic

Use `math.fma` where available. Provide a tested fallback. Use error-free `TwoSum/TwoProd` or compensated summation for:

- length tree aggregates;
- cumulative parameter weights;
- coefficient sums with severe cancellation;
- global displacement verification.

16.4 Underflow and overflow

Patch normalization SHALL keep solver arithmetic away from subnormal and overflow ranges. `ldexp` SHALL be used to restore physical scales. Every restoration SHALL check finiteness.

The target test range SHALL include coordinate scales from at least `1e-150` through `1e307`, while recognizing that a tiny displacement superimposed on a huge coordinate may be unrepresentable in binary64 and must be rejected rather than fabricated.

16.5 Small-angle formulas

Any construction formula involving

$$\frac{\sin x}{x}, \quad \frac{1 - \cos x}{x^2}, \quad \frac{e^{ix} - 1}{x}$$

SHALL use stable `sinc`-style series or half-angle forms near zero.

16.6 Determinism

Given identical binary64 inputs, policies, package version, NumPy/SciPy major versions, and execution architecture, construction SHALL be deterministic. Thread-count-dependent reductions SHALL be avoided in acceptance-critical code.

16.7 High-order geometry evaluation

Each derivative and curvature-vector query SHALL carry a conservative componentwise forward-error estimate assembled from coefficient extraction, local evaluation, series operations, affine parameter scaling, and physical-frame restoration. The bound SHALL grow explicitly with degree, requested order, recurrence depth, and the condition estimates of B^{-1} . It MUST NOT be a fixed multiple of machine epsilon independent of those quantities.

Before evaluating a jet, scale the preimage coefficients by a common power of two so their maximum finite magnitude lies in a safe normal range. Perform Bernstein differences, products, and reciprocal recurrences on the scaled coefficients; propagate the removed exponent analytically. Cancellation-sensitive complex products SHALL use componentwise FMA formulas or error-free product expansions. At a join, left/right agreement SHALL compare the difference against the sum of the two forward-error bounds plus the continuity verification bound appropriate to the order.

The evaluator SHALL return only finite binary64 components. If a mathematically nonzero component is outside the representable range, the reciprocal-speed recurrence loses certification, or the propagated uncertainty cannot distinguish two discontinuous one-sided values, it raises `NumericalPrecisionError` or `DiscontinuousDerivativeError` as applicable. It SHALL never return NaN, infinity, a silently saturated value, or a fabricated zero. Exact structural zeros and polynomial derivatives above degree are returned as exact zero vectors.

17. Verification pipeline

17.1 Independent verification principle

Construction and verification SHALL use at least partially independent code paths. Reusing the exact same residual function and declaring it verified is insufficient.

Recommended independent pairs:

- solver displacement in B-spline/extraction form; verifier displacement from compiled Bezier antiderivative controls;
- solver continuity from shared B-spline controls; verifier continuity from left/right local jets;
- speed coefficients from Bernstein products; verifier speed from direct preimage evaluation at certified points plus coefficient identity;
- inverse LUT construction from robust root solves; runtime verification from direct forward/reverse arc evaluation.

17.2 Required constructor checks

Before publication, verify:

1. every input point is reached at its interpolation knot within the position bound;
2. every displacement constraint passes independently;
3. every span PH reconstruction identity passes;
4. every requested C, G, and curvature continuity condition passes;
5. every span is regular with the required margin;
6. every span length is positive and finite;
7. the length tree total equals an independent compensated span sum;
8. every inverse LUT is strictly monotone and bracket-correct;
9. representative and adversarial local inverse targets pass the residual bound;
10. all public endpoint values are finite;
11. closed curves pass seam position and continuity checks.

17.3 Position tolerance

For a patch scale H and coordinate value P , define a conservative position bound

$$\tau_P = 256\epsilon H + 8 \max(\text{ulp}(P_x), \text{ulp}(P_y)) + E_{\text{eval}}.$$

The exact factor SHALL be centralized and tested. A fixed absolute tolerance such as $1\text{e-}9$ is forbidden.

17.4 Continuity tolerance

For a physical jet J , use a combined bound

$$\tau_J = C_J \epsilon \max(1, \|J_-\|, \|J_+\|) + E_{J,-} + E_{J,+},$$

where C_J grows conservatively with derivative order and polynomial degree. High-order continuity requests may be rejected if the bound becomes too large to be meaningful.

The default tangent absolute gate SHALL not exceed $1\text{e-}12$ for well-conditioned normalized data. Curvature comparison SHALL use relative scaling and default behavior comparable to $1\text{e-}10$ on unit-scale data.

17.5 Verification report

BuildDiagnostics and EditReport SHALL include:

- solver iterations;
- refinement rounds and hidden spans inserted;
- maximum interpolation residual and bound;
- maximum C/G/curvature continuity residual and bound;
- minimum regularity ratio;
- maximum inverse residual ratio;
- LUT sizes and fast-path correction counts;
- patch span count for edits;
- whether long-double verification was used;
- warnings that did not invalidate the result.

18. Exceptions

18.1 Hierarchy

All package exceptions SHALL derive from `PHBSplineError`. Input/query errors SHALL also derive from `ValueError`; numerical construction/edit/query failures SHALL also derive from `RuntimeError`.

Required hierarchy:

```
PHBSplineError
+-- PHBSplineValueError, ValueError
|   +-- InvalidPointDataError
|   +-- InsufficientPointDataError
|   +-- NonFiniteCoordinateError
|   +-- DegenerateConsecutivePointError
|   +-- ContinuitySpecificationError
|   +-- ParameterOutOfRangeError
|   +-- ArcLengthOutOfRangeError
|   +-- UndefinedPrincipalNormalError
|   +-- DiscontinuousDerivativeError
|   +-- StaleHandleError
|   +-- StaleLocationError
+-- PHBSplineRuntimeError, RuntimeError
    +-- ConstructionConvergenceError
    +-- InterpolationVerificationError
    +-- ContinuityVerificationError
    +-- NonRegularSplineError
    +-- ArcLengthInversionError
    +-- LengthResolutionError
    +-- NumericalPrecisionError
    +-- LocalEditFailure
    +-- ResourceLimitError
    +-- TransactionError
```

18.2 Structured fields

Every package exception SHALL accept and expose, as applicable:

```
index: int | tuple[int, ...] | None
point_id: int | None
span_id: int | None
operation: str | None
patch: tuple[int, int] | None
quantity: str | None
value: object
bound: object
iteration: int | None
```

The formatted message SHALL include populated fields. Original fields remain machine-readable.

19. Complexity requirements

Let:

- N be user point count;
- M be compiled PH span count including hidden spans;
- m be preimage degree;
- K be edited patch span count;
- I be bounded nonlinear iterations.

For fixed m , bounded hidden-span ratio, and bounded I :

Operation	Required complexity
Initial validation and guide	$O(N)$
Initial construction	$O(N)$ per bounded iteration; overall $O(N)$ under caps
Scalar <code>point(u)</code>	$O(\log M + C_{\text{eval}}(m))$
Scalar <code>arc_length(u)</code>	$O(\log M + C_{\text{eval}}(m))$
Random <code>point_at_length(s)</code>	$O(\log M + C_{\text{eval}}(m) * I_{\text{inv}})$ with bounded I_{inv}
Sorted batch distance queries	$O(M + Q * C_{\text{eval}}(m))$ or $O(Q * C_{\text{eval}}(m))$ after the starting span
Strict-local move/insert/delete	$O(\log N + K * m^3 * I)$, with bounded K
Tree metric update	$O(\log M)$
Compact snapshot build	$O(M)$
Sequential cursor advance	amortized $O(1 + \text{crossed_spans})$

Absolute timing targets SHALL be benchmark gates, not API guarantees. The reference target for default G2 on a modern desktop is:

- mutable scalar random `point_at_length`: median below 15 microseconds for 100 to 100,000 spans;
- compact snapshot scalar random `point_at_length`: median below 8 microseconds where Python dispatch dominates;
- one-point strict-local move: latency approximately independent of total N once N exceeds leaf/patch size;
- no more than two Halley/Newton corrections for at least 99.9 percent of benchmark distance queries; all remaining queries use the bounded fallback.

These numbers SHALL be measured on a declared platform and MUST NOT be presented as universal guarantees.

20. Serialization

20.1 Authoritative state

Serialization SHALL include:

- package format version;
- user points and stable point IDs;
- closed/open flag;
- continuity request and policies;
- knot topology and hidden-span mapping;
- preimage coefficients or authoritative local span representation;
- verification metadata;
- optional compiled inverse kernels and tree aggregates.

20.2 Loading

On load, the implementation SHALL validate schema, finiteness, array sizes, and checksums. By default it SHALL perform lightweight structural and metric verification before exposing the object. A `verify="full"` mode SHALL rerun all postconditions.

Untrusted serialized data MUST NOT be allowed to bypass regularity or bounds checks.

21. Threading and concurrency

The mutable `PHBSpline` SHALL support multiple concurrent readers and a single writer through a read/write lock or immutable-root publication.

- Queries observe one complete version.
- An edit constructs a private candidate and publishes it atomically.
- Query methods SHALL NOT observe partially updated length aggregates or span caches.
- `PHBSplineSnapshot` SHALL be lock-free for reads after construction.
- User callbacks SHALL not be executed while internal locks are held.

22. Testing specification

22.1 Baseline test migration

The new suite SHALL port the intent of every `cubic-ph-spline` test category:

- arc-length evaluation and inversion;
- classification and nonconvex geometry;
- extreme scales;
- frames and normals;
- continuity;
- geometry and endpoint interpolation;
- invariants;
- straight-line cases;
- validation and exceptions.

The existing retrieved baseline passed 491 tests; the new implementation SHALL not reduce adversarial coverage merely because the representation changed.

22.2 Input geometry matrix

Tests SHALL include:

- random convex polygons;
- highly nonconvex open polylines;
- many alternating inflections;
- self-intersections and figure-eight data;
- exact and near 180-degree reversals;
- repeated nonconsecutive points;
- open and closed loops;
- long straight runs mixed with tight turns;

- chord ratios down to the representability/regularity guard;
- 100,000-point data streams.

For any configuration that cannot be constructed under policy limits, the test SHALL verify the exact exception type, structured fields, and transactional rollback.

22.3 Continuity matrix

At minimum test:

- G0 through G5;
- C0 through C5;
- curvature C0 through C3;
- mixed requests such as G2+C1, C3+curvature C1, and G4+curvature C2;
- open boundaries and closed seam continuity.

Verify left/right jets independently in normalized and physical coordinates.

22.4 Dynamic-edit property tests

For point counts including 10, 1,000, and 100,000:

- move interior and endpoint points;
- insert before, after, and within leaf boundaries;
- delete interior and endpoint points;
- perform long randomized edit sequences;
- batch edits in transactions;
- induce deliberate strict-local failure and verify exact rollback;
- compare an expanded/global rebuild with a fresh constructor build under the same deterministic policy;
- verify unchanged spans outside the patch are the identical objects when bitwise preservation is enabled;
- verify all retained point handles remain valid and unchanged;
- verify deleted handles become stale only after commit.

22.5 Distance inversion tests

For every tested span, query:

- 0, L, and the midpoint;
- one and several ULPs from both endpoints;
- every LUT node and cell midpoint;
- random uniform distances;
- distances concentrated where speed is minimal;
- spans at the regularity-ratio threshold;
- alternating tiny and huge span lengths;
- targets exactly at tree prefix boundaries;
- scalar float and `LengthCoordinate` forms;
- sorted and unsorted batch forms.

Acceptance SHALL verify:

$$|S(\nu) - s| \leq \tau_s,$$

monotonicity, bracket preservation, and absence of warnings/NaNs.

22.6 Arbitrary-order geometry queries

For parameter derivatives, arc-length derivatives, scalar curvature derivatives, and curvature-vector derivatives, test every order from zero through at least 12 and selected orders through the configured default limit. Include endpoints, exact and near joins, inflections, certified straight spans, minimum-speed locations, alternating coefficient scales, and

orders above the polynomial degree. Verify `side="auto"`, explicit one-sided values, and the required discontinuity exception at joins whose available continuity is too low.

Check the identities

$$D_s z = T, \quad D_s^2 z = \kappa N_L, \quad D_s^j(\kappa N_L) = D_s^{j+2} z$$

against independently evaluated left/right jets and high-precision references. Compare the single-order methods with their full-jet counterparts, require exact structural zeros where specified, and exercise invalid types, negative orders, resource-limit rejection, and output overflow/underflow. Finite-difference comparisons MAY be used as low-precision smoke tests only; they are not acceptance oracles.

22.7 High-precision oracle tests

An optional test dependency such as `mpmath` SHALL evaluate selected construction, arc-length, derivative-vector, scalar-curvature, curvature-vector, and inverse cases at 100 or more decimal digits. Production code SHALL not depend on this package.

22.8 Scale and transformation invariance

Apply translations, rotations, reflections, and power-of-two scalings across at least `1e-150` through `1e307` where the transformed coordinates remain representable.

Verify:

- positions transform correctly;
- tangents rotate/reflect correctly;
- parameter derivative vectors transform with the spatial scale, and arc-length derivative order r scales as $|\lambda|^{1-r}$ with the corresponding orthogonal vector transform;
- lengths scale by absolute scale;
- curvature scales inversely;
- the j th arc-length derivative of the curvature vector scales as $|\lambda|^{-j-1}$ with the corresponding orthogonal vector transform;
- distance inversion returns corresponding locations;
- edit locality and handle identity remain unchanged.

22.9 Fuzzing

Use property-based testing where practical. Fuzz:

- point arrays and malformed inputs;
- continuity values;
- edit sequences;
- tree split/merge boundaries;
- extreme distance values;
- derivative orders, `wrt` modes, and join-side selections;
- LUT sizes;
- serialized data corruption.

No fuzz input may produce an unhandled `RuntimeWarning`, segmentation fault, infinite loop, or untyped exception.

22.10 Performance regression tests

Benchmarks SHALL measure:

- construction time versus `N` at fixed order;
- strict-local edit time versus `N` at fixed patch size;
- tree update time;
- scalar random distance access versus `N`;
- compact snapshot versus mutable tree queries;

- sorted batch throughput;
- scalar and batch derivative-vector and curvature-vector throughput versus requested order;
- full-jet throughput versus repeated single-order calls;
- correction-count distribution;
- memory per input point and per hidden span.

The edit benchmark SHALL explicitly demonstrate flat local solve time plus logarithmic tree overhead from 1,000 to 100,000 points.

23. Acceptance criteria

Version 1 is releasable only when all of the following are true:

1. The documented public API and exception hierarchy are implemented and typed.
2. Default construction of representative arbitrary point data produces a verified regular G2 spline.
3. All accepted curves satisfy exact point interpolation within the scale-aware bound.
4. Requested finite continuity is verified at every join and closed seam.
5. Arbitrary-order derivative-vector and curvature-vector queries use the specified elementary kernels, satisfy their jet identities and error bounds, and never use finite differences.
6. Arc length is generated analytically from PH coefficients with no quadrature.
7. Every scalar inverse query either meets the residual bound or raises a typed exception.
8. Strict-local edits are atomic, leave exterior geometry unchanged, and have measured runtime independent of total point count apart from tree lookup/update.
9. Expanding/global repair behavior is explicit and tested.
10. 100,000-point construction, query, and local-edit tests pass within declared memory limits.
11. Scale, reversal, inflection, self-intersection, and near-regularity adversarial tests pass or fail with the declared typed exceptions.
12. The package emits no `RuntimeWarning` under the test suite.
13. Documentation does not claim universal existence, exact symbolic high-degree inversion, or unconditional successful $O(\log N)$ edits.

24. Implementation sequence

The recommended work breakdown is:

Phase 1 - immutable mathematical core

- Bernstein and B-spline extraction utilities;
- variable-degree preimage and span compilation;
- exact speed and arc polynomial generation;
- arbitrary-order derivative and curvature-vector kernels;
- geometry evaluation and regularity certificate;
- immutable `SpanKernel` tests.

Phase 2 - distance kernel

- double-double length arithmetic;
- forward/reverse arc evaluators;
- LUT construction;
- safeguarded inverse solver;
- scalar and batch distance tests.

Phase 3 - static PH B-spline constructor

- guide initializer;
- analytic displacement constraints and Jacobian;
- deterministic sparse/banded solver;

- hidden-knot refinement;
- full independent verification;
- immutable snapshot API.

Phase 4 - dynamic tree and local edits

- point/span B+ tree;
- stable handles and locations;
- transactional move;
- insertion and deletion;
- persistent snapshots;
- strict-local and expanding policies.

Phase 5 - enterprise hardening

- extreme-scale local frames;
- serialization;
- concurrency;
- fuzzing and high-precision oracles;
- 100,000-point benchmarks;
- documentation and migration guide from CubicPHSpline.

No phase should introduce a public unverified fallback merely to keep the API running.

25. Example usage

```
import numpy as np
from ph_spline import PHBSpline

points = np.array(
    [
        [0.0, 0.0],
        [1.0, 0.4],
        [2.0, -0.7],
        [3.0, 1.1],
        [2.2, 2.0],
    ],
    dtype=np.float64,
)

# With no explicit continuity arguments, the guarantee defaults to G2.
curve = PHBSpline(points)

L = curve.length
p = curve.point_at_length(0.37 * L)
u = curve.parameter_at_length(0.37 * L)
t = curve.tangent(u)

# Elementary arbitrary-order geometry kernels; no numerical differencing.
d5 = curve.derivative(u, 5, wrt="parameter")
k3 = curve.curvature_vector(u, 3) # D_s^3(kappa * N_left) = D_s^5 z
z_jet = curve.jet(u, 6, wrt="arc_length")
k_jet = curve.curvature_vector_jet(u, 4)

# Stable handle survives insertion and index shifts.
h = curve.point_handle(2)
report = curve.move_point(h, [2.1, -0.9], repair="strict_local")
```



```

inserted = curve.insert_point(3, [2.7, 0.2])
curve.delete_point(inserted.handle)

# Local relative travel avoids loss of global distance resolution.
loc = curve.location_at_length(0.5 * curve.length)
loc2 = curve.advance_by_length(loc, 1.0e-6)
p2 = curve.point_after_length(loc, 1.0e-6)

# Higher-order request. Degree is deduced, not fixed in the class.
curve_c4 = PHBSpline(points, c_order=4, curvature_order=2)
assert curve_c4.preimage_degree == 4
assert curve_c4.degree == 2 * curve_c4.preimage_degree + 1

# Lock-free query object.
snapshot = curve.snapshot(compact=True)
positions = snapshot.points_at_length(np.linspace(0.0, snapshot.length, 10000))

```

26. Required pseudocode

26.1 Scalar distance query

```

function location_at_length(s):
    target = validate_length_coordinate(s)
    if target == 0: return first span at u=0
    if target == total_length: return last span at u=1

    node, local_s = length_tree.predecessor_and_remainder(target)
    span = node.span
    L = span.length_dd

    if local_s <= L / 2:
        direction = FORWARD
        target_local = local_s
        xi = target_local / L
    else:
        direction = REVERSE
        target_local = L - local_s          # double-double subtraction
        xi = target_local / L

    cell = min(floor(xi * span.lut.M), span.lut.M - 1)
    lo, hi = span.lut.bracket(cell, direction)
    u = span.lut.seed(cell, xi, direction)
    u = clamp(u, lo, hi)

    for iteration in 0 .. fast_iterations-1:
        f, fp, fpp, eval_bound = span.arc_residual_derivatives(u, target_local, direction)
        if abs(f) <= inverse_tolerance(L, target_local, eval_bound):
            return location(span, recover_direction(u), version)
        update bracket from sign(f)
        proposal = safeguarded_halley_or_newton(...)
        if proposal invalid: break
        u = proposal

    for iteration in fast_iterations .. max_iterations-1:
        use ITP/bisection/Newton while preserving bracket
        verify residual after each evaluation

```

```
raise ArcLengthInversionError(...)
```

26.2 Strict-local move

```
function move_point(handle, new_point, repair="strict_local"):
    begin private transaction
    resolve handle and validate new_point
    modify candidate interpolation record
    patch = initial_support_patch(handle, degree, hidden_spans)

    while true:
        freeze exterior preimage controls and boundary jets
        build normalized candidate guide in patch
        solve changed displacement constraints with analytic Jacobian
        compile candidate span kernels
        independently verify interpolation, continuity, regularity, and inverse kernels

        if verified:
            replace affected persistent tree path
            update length and parameter aggregates
            atomically publish root and version
            return EditReport

        if repair == "strict_local":
            rollback and raise LocalEditFailure
        if repair == "expand" and patch can expand:
            patch = expand_geometrically(patch)
            continue
        if repair == "global":
            rebuild all
            continue

    rollback and raise LocalEditFailure
```

27. Design rationale and nonclaims

27.1 Why quintic is the default

A degree-1 complex preimage produces a cubic PH curve whose signed curvature numerator is constant, so a regular nonstraight cubic PH segment cannot contain an inflection. A degree-2 preimage produces a quintic PH curve with enough local shape freedom to support inflections while retaining low-degree speed and arc polynomials. Default G2 therefore maps naturally to $m = 2$, degree 5.

27.2 Why the inverse is numerical despite polynomial arc length

For a degree- m preimage, the arc polynomial has degree $2m + 1$. Generic quintic and higher polynomials have no universal radical inverse. Restricting every speed polynomial to a radical-invertible compositional family would materially reduce planar shape freedom.

The specified LUT plus safeguarded monotone correction retains the general PH B-spline space while giving predictable, verified machine-precision queries. It is also likely to be faster than evaluating multiple transcendental special functions for many spans.

27.3 Why local support needs patch closure

Compact support of the preimage affects the hodograph locally, but position is its integral. Without a zero net displacement change across an edited patch, all downstream positions would translate. The fixed boundary positions

and patch displacement constraints are therefore part of the local-support definition, not an optional refinement.

27.4 Why strict locality can fail

A large point move can make the fixed exterior jets incompatible with any regular PH curve inside a small patch. No implementation can guarantee both successful arbitrary edits and an immutable finite patch in all cases. The API exposes the honest alternatives:

- strict local and transactional failure;
- expanding patch with potentially larger cost;
- explicit global rebuild.

28. References

1. H. Abraham, [cubic-ph-spline](#), version 1.1.0 metadata and repository documentation, reviewed 2026-08-05.
2. G. Albrecht, C. V. Beccari, J.-C. Canonne, and L. Romani, “Pythagorean-Hodograph B-Spline Curves”, *Computer Aided Geometric Design* 57 (2017), 57-77, DOI 10.1016/j.cagd.2017.09.001.
3. R. T. Farouki, [Introduction to Pythagorean-Hodograph Curves](#), Springer-related author manuscript/material.
4. C. Giannelli, L. Sacco, and A. Sestini, “A local C2 Hermite interpolation scheme with PH quintic splines for 3D data streams”, 2021.
5. M. Knez, F. Pelosi, and M. L. Sampoli, “Construction of G2 planar Hermite interpolants with prescribed arc lengths”, 2022.
6. J. Kosinka and M. Lavicka, “Pythagorean Hodograph Curves: A Survey of Recent Advances”, *Journal for Geometry and Graphics* 18(1), 2014.

Appendix A. Suggested diagnostic dataclasses

```
@dataclass(frozen=True, slots=True)
class ContinuitySpec:
    g_order: int | None
    c_order: int | None
    curvature_order: int | None

@dataclass(frozen=True, slots=True)
class BuildDiagnostics:
    point_count: int
    span_count: int
    hidden_span_count: int
    preimage_degree: int
    iterations: int
    refinement_rounds: int
    max_interpolation_residual: float
    interpolation_bound: float
    max_continuity_residual: float
    continuity_bound: float
    min_regularity_ratio: float
    max_inverse_residual_ratio: float
    max_lut_nodes: int
    longdouble_verification_used: bool

@dataclass(frozen=True, slots=True)
class EditReport:
    operation: str
    version_before: int
    version_after: int
    affected_point_ids: tuple[int, ...]
    affected_span_ids: tuple[int, ...]
```

```

rebuilt_span_count: int
patch_span_count: int
iterations: int
refinement_rounds: int
hidden_spans_added: int
max_interpolation_residual: float
max_continuity_residual: float
min_regularity_ratio: float

@dataclass(frozen=True, slots=True)
class InsertResult:
    handle: PointHandle
    report: EditReport

@dataclass(frozen=True, slots=True)
class Frame2D:
    point: NDArray[np.float64]
    tangent: NDArray[np.float64]
    left_normal: NDArray[np.float64]
    signed_curvature: float

```

Appendix B. Double-double primitives

The length tree SHOULD use standard error-free transformations:

```

TwoSum(a, b):
    s = a + b
    bb = s - a
    e = (a - (s - bb)) + (b - bb)
    return s, e

```

```

FastTwoSum(a, b), requiring |a| >= |b|:
    s = a + b
    e = b - (s - a)
    return s, e

```

Double-double addition SHALL renormalize the pair. Tree comparisons SHALL compare the exact pair lexicographically only after accounting for overlapping low parts; a tested helper type is required rather than ad hoc tuple arithmetic.

Appendix C. Formal derivative and curvature-vector jet recurrence

Let $w(t)$ be represented by normalized truncated power-series coefficients around a query or join. Construct series for:

$$A(t) = \text{Im}(\bar{w}w'), \quad B(t) = |w|^2.$$

Then

$$\kappa(t) = 2A(t)B(t)^{-2}.$$

The curve parameter derivatives follow directly from

$$z'(t) = w(t)^2,$$

and the curvature-vector series follows from the elementary complex expression

$$C_0(t) = 2A(t) i w(t)^2 B(t)^{-3}.$$

Use finite formal-series multiplication, reciprocal, and differentiation to obtain the parameter derivatives. Compute one reciprocal series for B^{-1} and obtain B^{-2} and B^{-3} by multiplication; do not run independent divisions for each requested order. Convert any scalar or vector field F to arc-length derivatives recursively with

$$F_0 = F, \quad F_{j+1} = B^{-1} \frac{dF_j}{dt}.$$

Use $F = z$, $F = \kappa$, or $F = C_0$ for the curve derivative, scalar-curvature, or curvature-vector jet respectively. The recurrence applies componentwise to vector fields. Normalized Taylor coefficients SHALL be used throughout, and conversion to ordinary derivative values occurs once at the API boundary with checked exponent scaling. This elementary procedure, or an algebraically identical generated recurrence, SHALL be used in production and verification. A general-purpose automatic-differentiation dependency is unnecessary, and numerical differencing is not acceptable.

Appendix D. Benchmark reporting template

Every published benchmark SHALL report:

CPU model and clock policy
operating system
Python version
NumPy and SciPy versions
compiler/runtime details
thread environment variables
continuity request and actual degree
number of input points and hidden spans
regularity-ratio distribution
LUT-node distribution
warm-up procedure
number of repetitions and statistic reported
construction time
move/insert/delete latency
random point_at_length latency
sorted batch throughput
correction-count histogram
fallback count
peak resident memory

Benchmarks SHALL include both a mutable tree and a compact immutable snapshot. A query benchmark that omits residual verification does not measure the production contract.